

Advanced Computer Networks



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering
Indian Institute of Technology Bhubaneswar

Flow-Control and Congestion-Control



**DSSRG: Decentralized
Smart Systems Research
Group**

<https://sites.google.com/iitbbs.ac.in/dssrg>

Or Google dssrg iitbbs



Basic functions (Recap) -

- **TCP:**

- **Multiplexing:**
 - Support multiple processes
 - Done by Port numbers
- **Reliable transfer of data**
 - Automatically retransmits lost data
 - Sliding window ARQ
 - (Automatic Retransmission reQuest)



TCP - Basic functions Recap -

- **Flow Control -**

- Faster sender will not overwhelm slower receivers.
- Done by Advertised Window.

- **Congestion Control -**

- Controls the transmission rate of the source to achieve highest possible data transmission rate **without causing network congestion**



UDP: Basic functions

- **Multiplexing**
- Transmits data as soon as possible (will not delay transmission to improve efficiency)
- No reliability, or nothing
- No other service : **Very close** to that of the **IP protocol**
- Purpose: Invention of new Transport Protocol, Research, Fast prototyping....



Traffic Patterns on the Internet

- According to some measurements, the traffic pattern on the Internet is as follows:
- **75% of the transmissions are Web traffic**
(Note that **HTTPS** protocol data are transmitted using the **TCP** protocol)
- **90-95%** of these transmissions uses **TCP** - this includes the Web traffic because HTTP (an application layer) uses TCP (transport layer).
- Some use UDP - Live streaming, Online gaming, VoIP.
- Modern web traffic increasingly uses **QUIC** (Quick UDP Internet Connections), a protocol developed by Google that underlies HTTP/3, allowing secure and fast web browsing over UDP



What about rest 25% (may or may not be TCP)

- **Streaming Media:** Video and audio streaming (e.g., Netflix, YouTube, Spotify) that often bypasses traditional web browsers.
- **File Transfer/P2P:** Peer-to-peer applications, file sharing, and torrent traffic.
- **Gaming:** Real-time data exchanged between gaming clients and servers.
- **IoT & Background Traffic:** Data from smart devices, API calls, background OS updates, and malware traffic



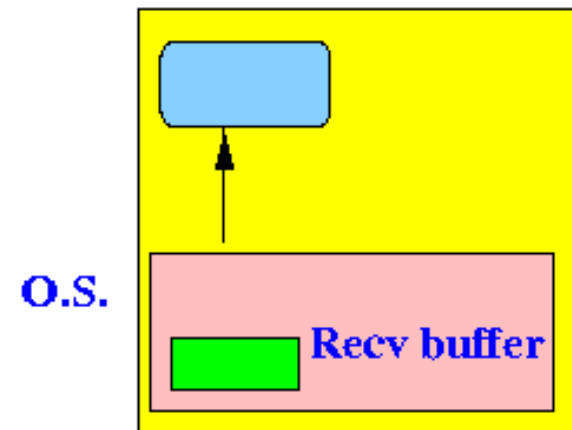
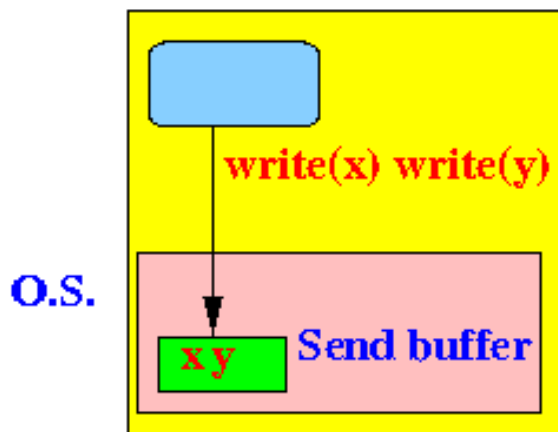
The moral of the story:

- TCP traffic has been (and probably will still be) a **dominant** portion of Internet traffic for the foreseeable future
- (because network applications that use UDP is very rare - only network research uses UDP).



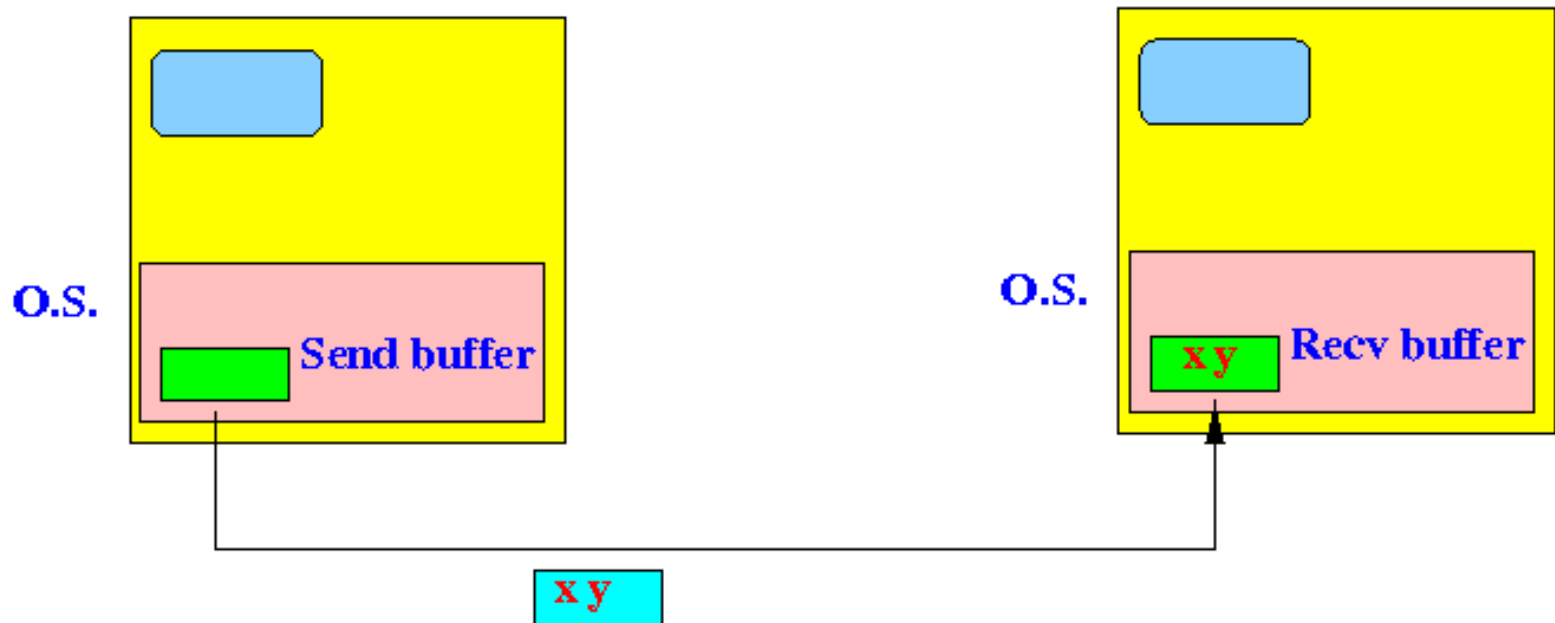
Flow control in TCP

- Send and Receive buffers
- When a **user application** sends data to another **user application**, the **data** is *first* stored in a *send buffer* inside Operating System kernel:
-

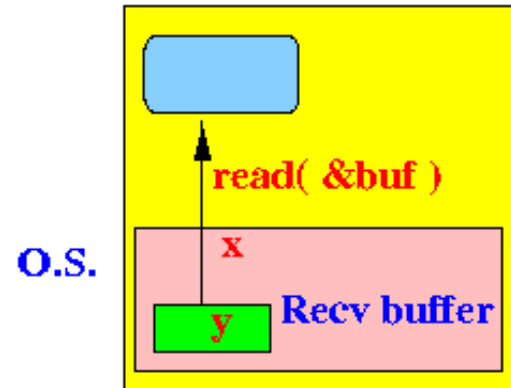
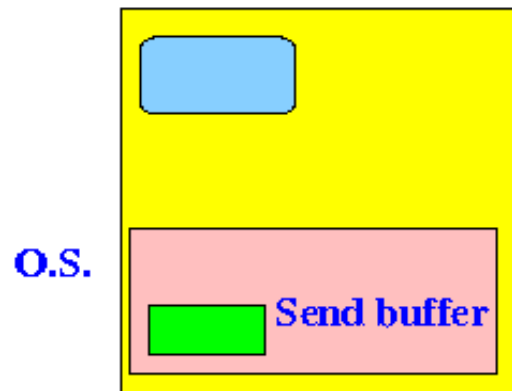


- When the **send buffer** contains **more than MSS bytes** or when the **send time *expires***, the **TCP module** will **send the data** in a **TCP segment** to the receiver:

-



- The **receive data** is stored in a **receive buffer** inside the Operating System kernel of the receiver computer
- The **receiving application** can use the **read()** system call to empty the **received data** from the **receive buffer**:
-



Flow Control

- **Flow control** is a **mechanism** to ensure that the **transmission rate** of the **sender** does *not* **exceed** the **capacity** of the **receiver**.
- If the transmission rate *exceeds* the capacity of the receiver, then sooner or later, there is no buffer space left to store arriving packets
- When this happens, newly arriving packets are discarded



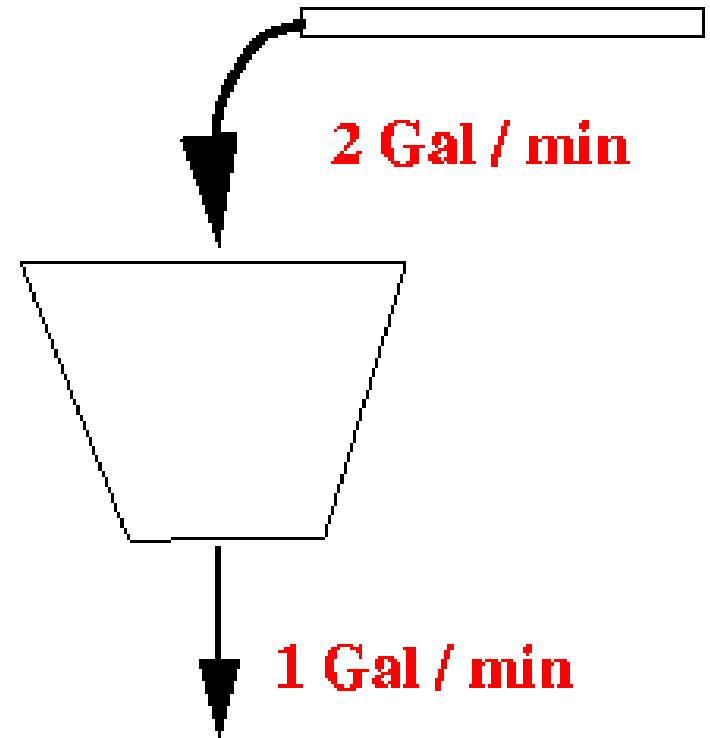
Analogy...

- Here's an analogy of what happens when the **receiver processing rate** is **lower** than the **transmission rate**:

The bucket has a hole that
let water drain at 1 Gal / min

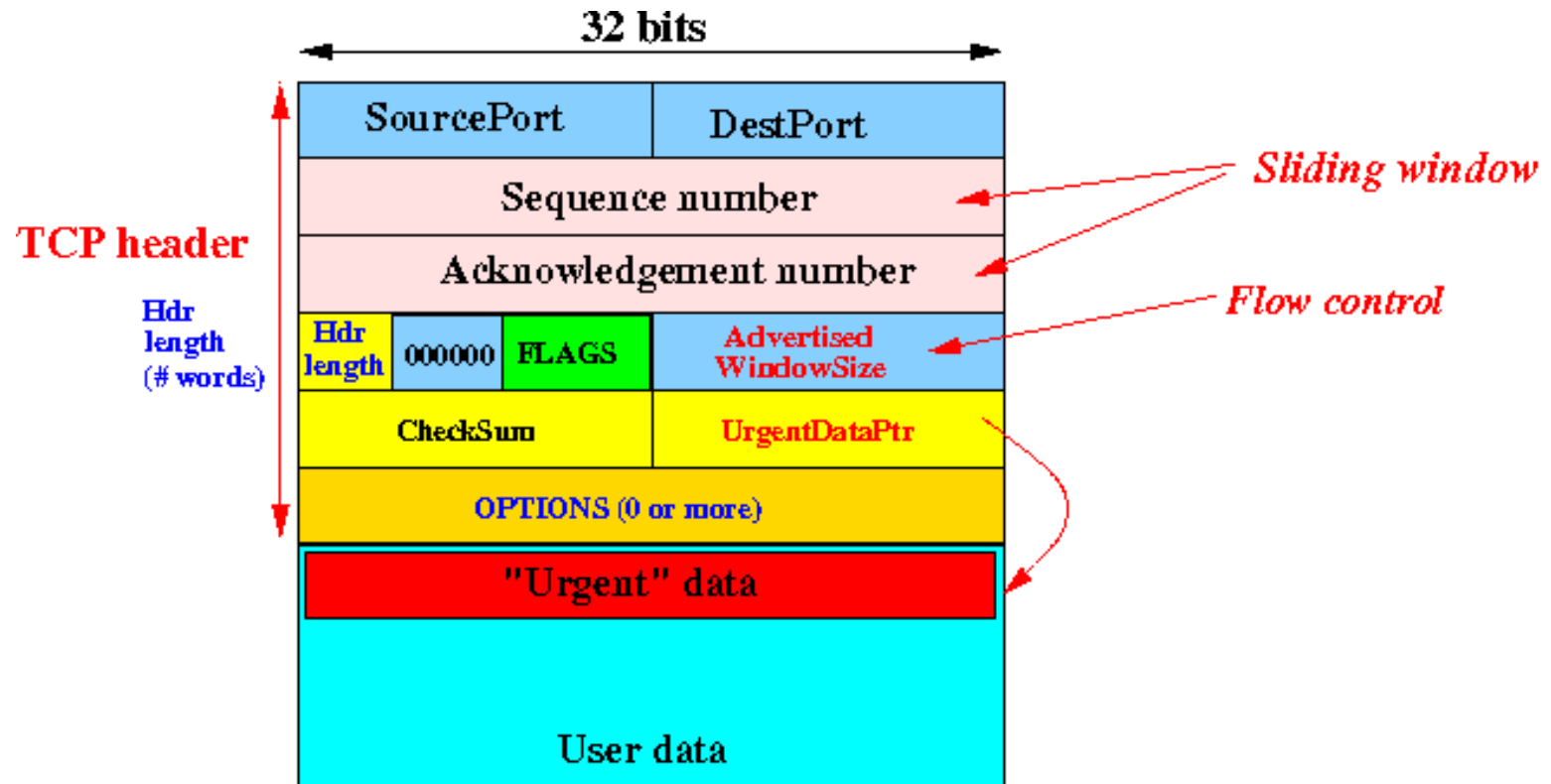
A pipe fills the bucket with water
at 2 Gal /min

Sooner or later, the bucket
will overflow



How is flow control implemented ?

- Flow control in TCP is implemented with the Advertised Window-Size (AWS) in the TCP header:



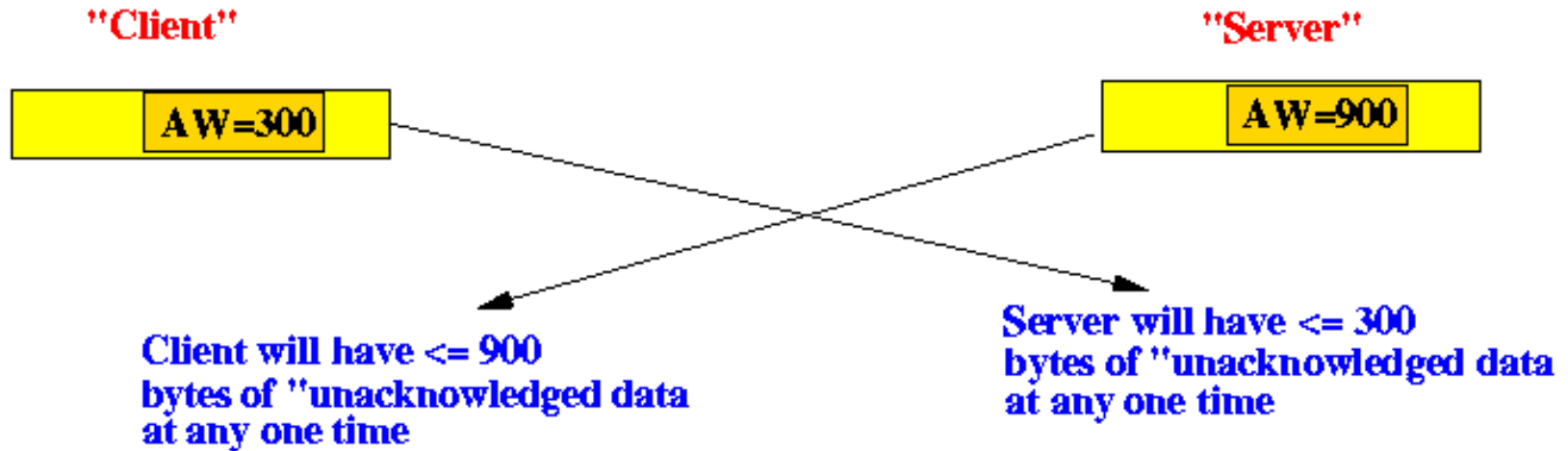
Using the Advertised Window Size:

- The **Adv. Win. size** is the **maximum number of bytes** that the **receiver** is willing to **buffer**

The **sender** should **honor** the **receiver's request** and **refrain from sending *more* data** than the given **adv. win. size**



Example



- The **client** tells the **server** *not* to send more than **300 bytes**
- The **server** tells the **client** *not* to send more than **900 bytes**

Receive buffer and Advertised Window Size

- **Factors** that determine the **Advertised Window Size**:
-

- The **size of the receive buffer** that the **TCP connection** has reserved –
- The **receive buffer size** is **fixed** at **connection establishment**
- The **amount of data** currently **buffered** in the **receive buffer**



The receive buffer:

- The **receive buffer** is located *inside* the **Operating System (kernel)**.
- The receive buffer is filled when a data segment is received
- The receive buffer is emptied (a little bit) when data is read by the user process
- The **user program** can set the **maximum receive buffer size** by using the **setsockopt()** system call



- From the **setsockopt** man page:
- The `SO_SNDBUF` and `SO_RCVBUF` options adjust the normal buffer sizes allocated for output and input buffers, respectively.
- The buffer size may be increased for high-volume connections or may be decreased to limit the possible backlog of incoming data.
- The maximum buffer size for UDP is determined by the value of the nnd variable `udp_max_buf`. The maximum buffer size for TCP is determined the value of the nnd variable `tcp_max_buf`....



- (Use **setsockopt** before you create a **socket**)
- The **default receive buffer size** in **Solaris** is **1 Mbytes**
- `$ ndd /dev/tcp tcp_max_buf 1048576`



What to store

- The following **types of data** can be stored in the **receive buffer**:
- (Type 1)

Deliverable data:

Data that has been **acknowledged** i.e., all **preceding data** have been **received**

The **application** can **readily** read the data (because the data can be delivered in the **order that was transmitted**)

- The following **types of data** can be store in the **receive buffer**:
(Type 2)

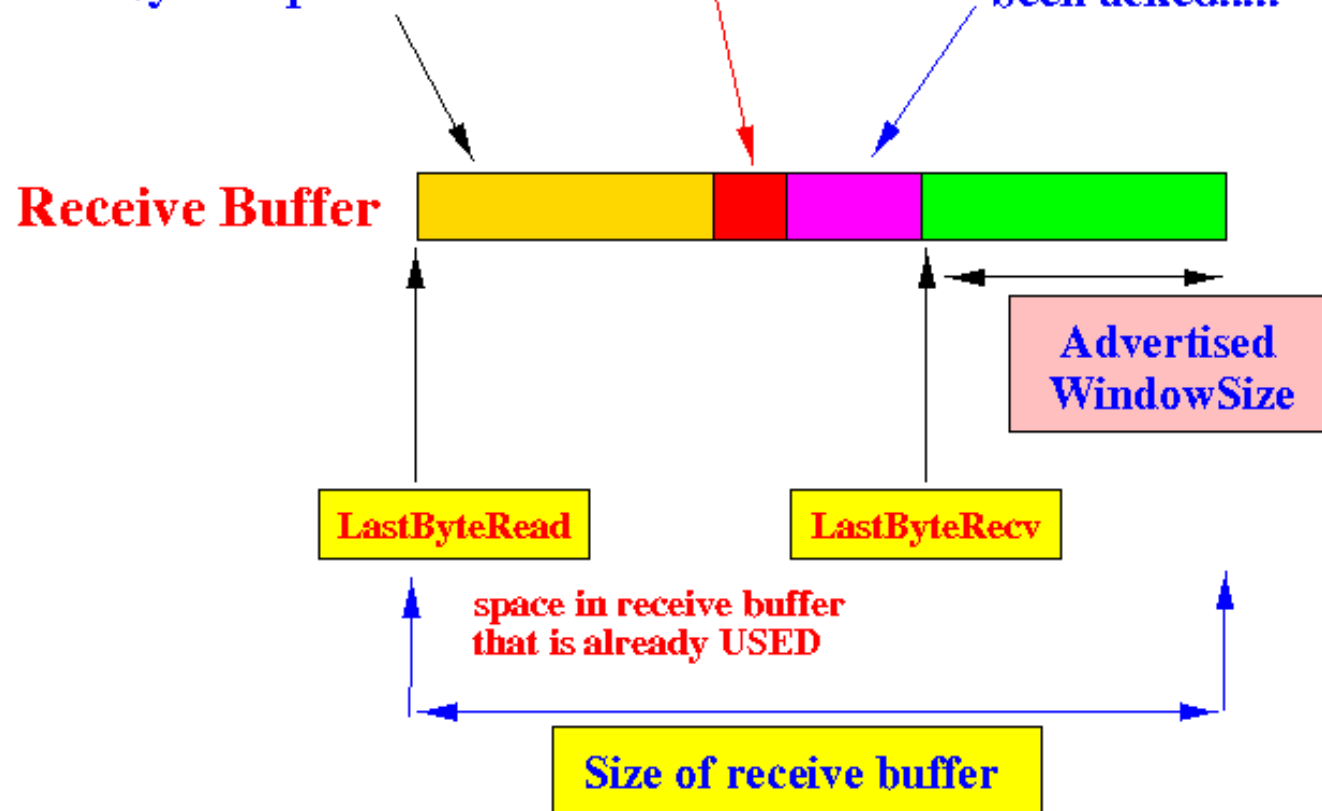
Undeliverable data:

Data that was received, but ***not*** been **acknowledged** because some **preceding data** has **not been received**

The **application cannot** read the data because the data ***would*** be read ***out of the order*** that was transmitted)

Computing the *Advertised Window Size*:

- Data received and Acked, but not yet read by user process
- Data was not received....
- Data was received and buffered, but has not been acked....



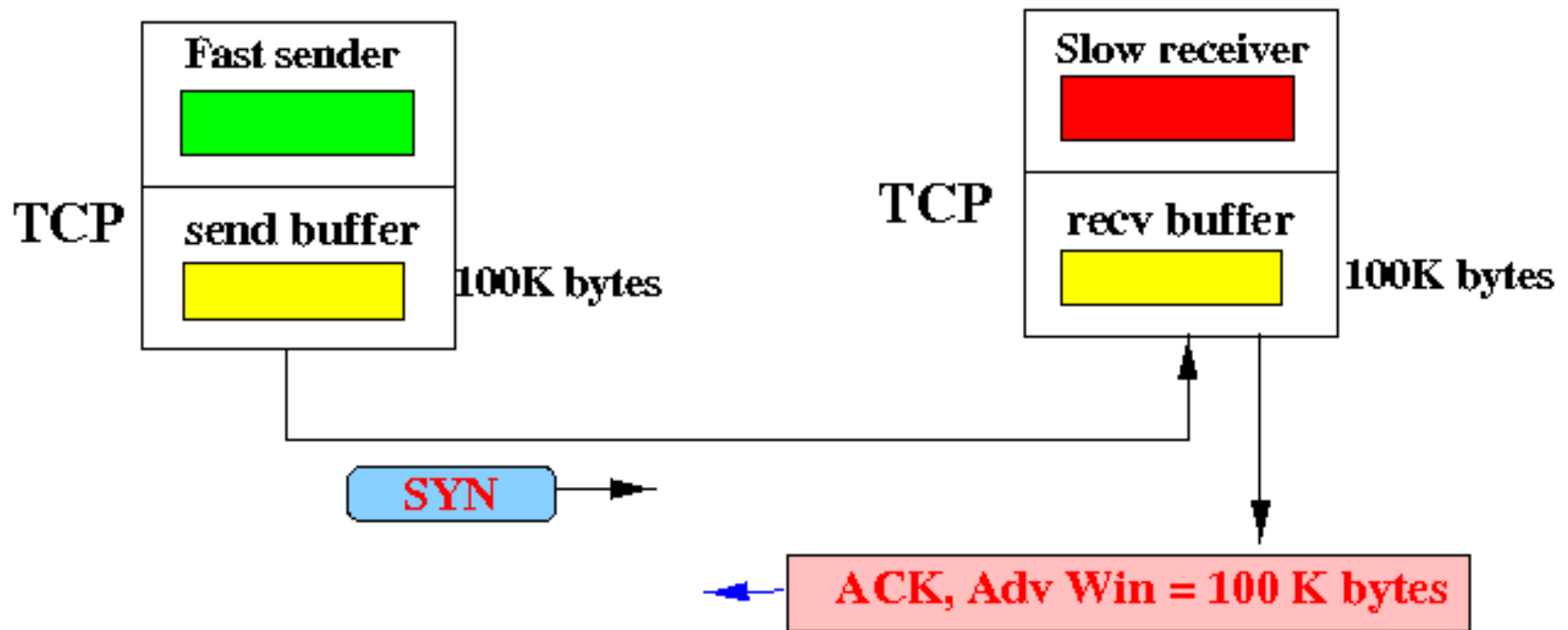
- Clearly, the **Advertised WindowSize** should be set to:
-

**Advertised Window = SizeRecvBuffer -
(LastByteRecv - LastByteRead)**

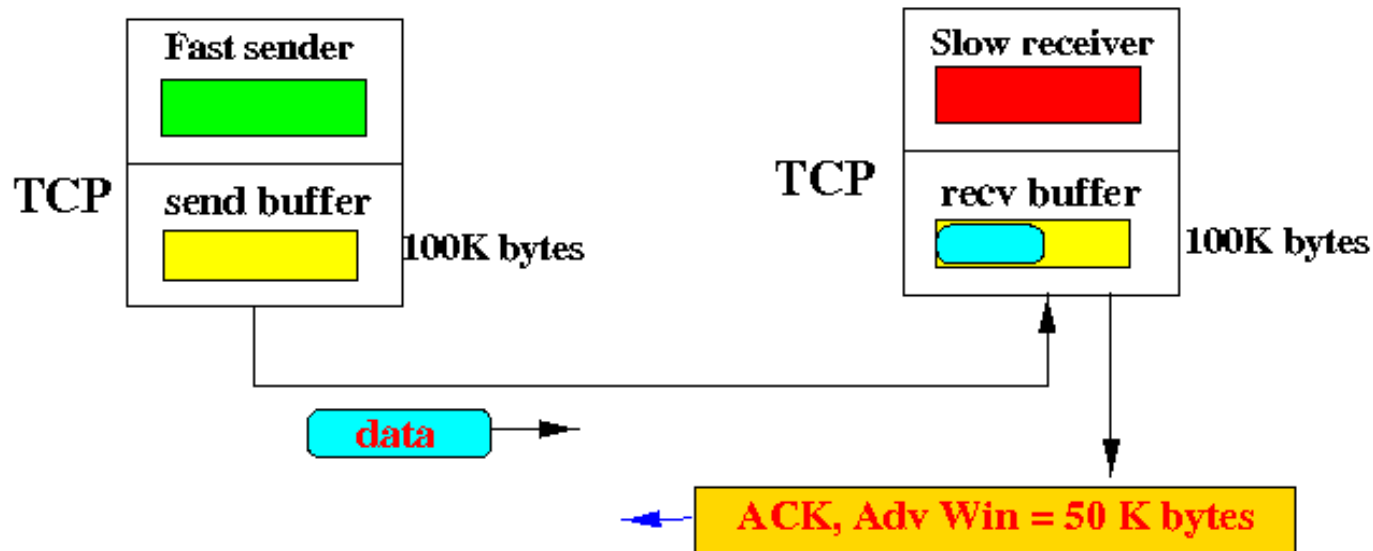
Flow control in action

- The **effect of flow control** is **only evident** when a **speedier sender** transmits to a **slower receiver !!!**
- The following example of **flow control** uses a **receive buffer size of 100 K bytes**.
- Initially, the send and receive buffer are empty.
- Receiver advertises a window of **100 K bytes**.

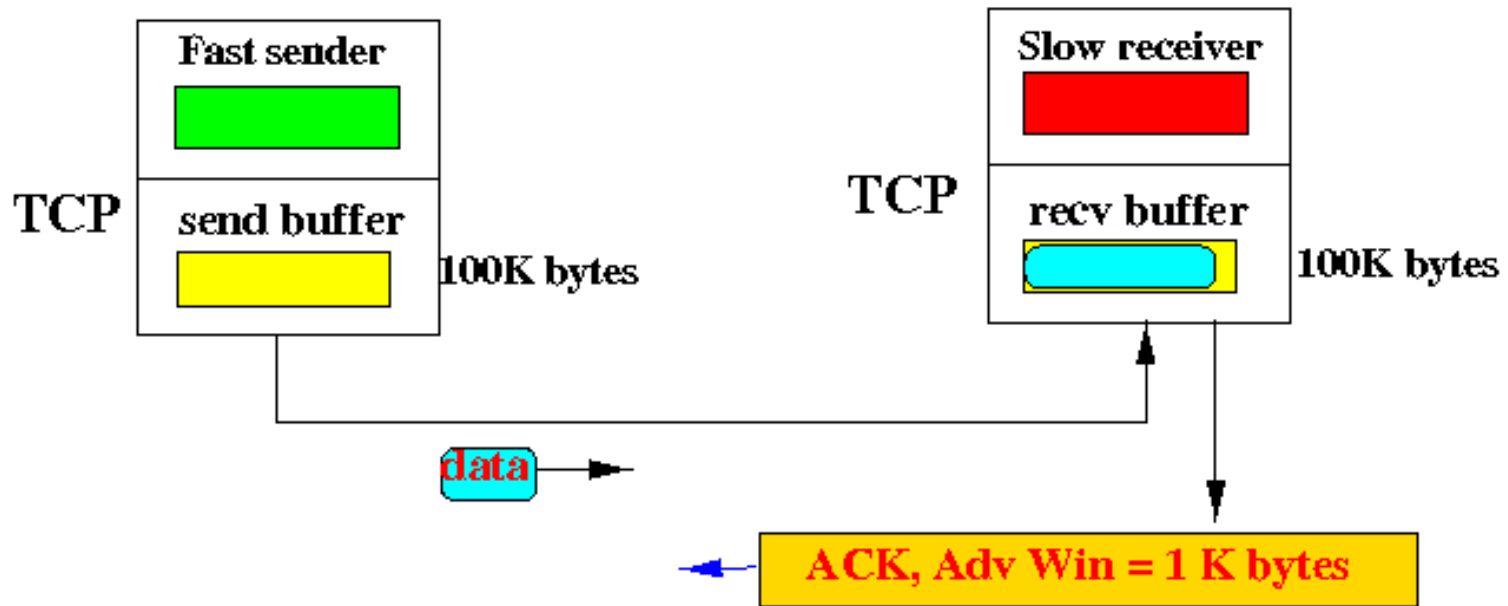




- Sender sends many packets quickly, faster than the receiver can process...
- the receive buffer starts to fill and soon the advertised window drops to **50 K bytes**:
-

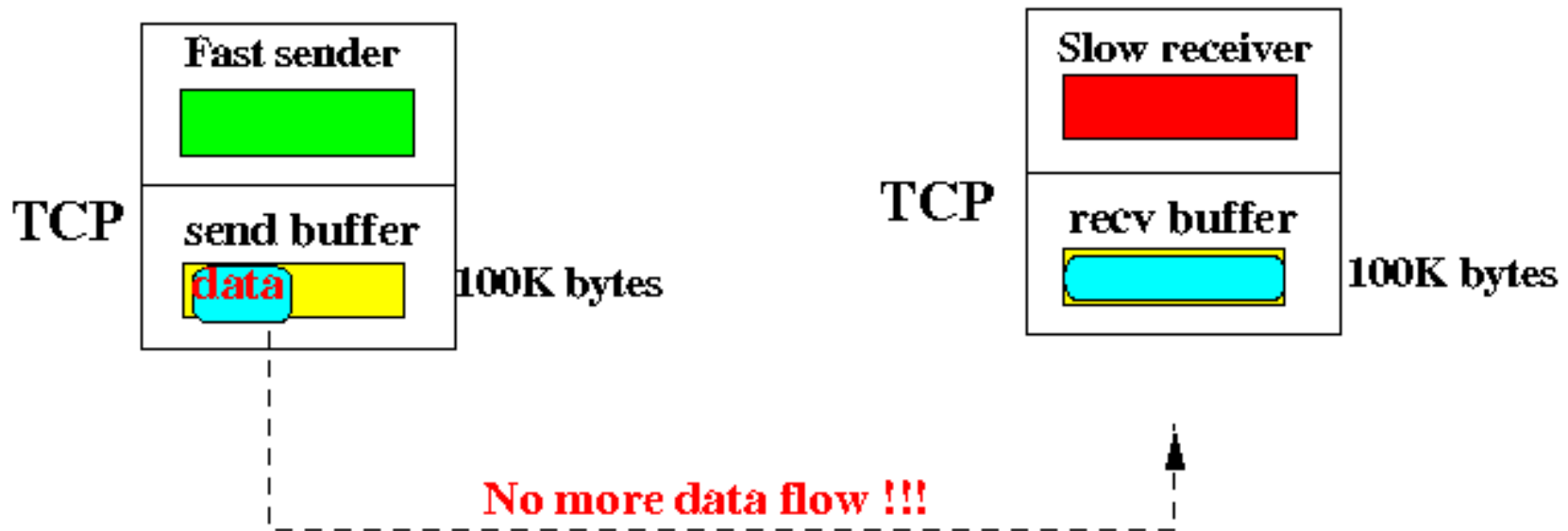


- The situation continues and the receive buffer fills further - The ACK packets from the receiver will now have a **lower advertised window**, e.g., 1 K bytes:



- Sooner or later, the **receive buffer fills up** - The receiver returns an ACK packets with the **advertised window = 0 bytes !!!**

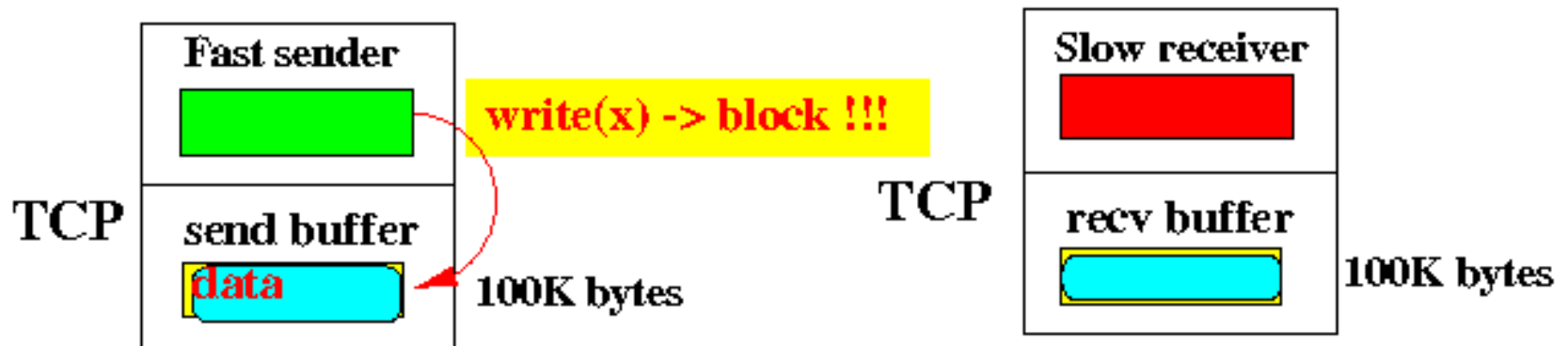
-



- This causes the sending TCP to stop transmitting more data..... and prevented the sending TCP to overflow the receive buffer of the receiving TCP.
- This will not stop the **sending application** from sending more data....The **sending application process** can still send more data...
- but the data sent will **remains in the send buffer !!!** (as given in the figure)
-



- If the **sending application process** continues sending, the **send buffer will fill up**
- A subsequent **write() call** will cause the **sending application process to block**:
-



- Now the **faster sending application process** has been **successfully throttled ...**
- **An advanced programming note**
- The **default behavior of write() is blocking**
- **Non-blocking write() operations:** - There are commands to **change ...**
- It will return -1 ...



The Blocked TCP flow problem...

- A receiving TCP protocol module will send an (ACK) packet only if it has *received* a TCP segment ...
- Rephrased:
- A receiving TCP protocol module will *not* send an (ACK) packet if it has *not* receive any TCP segment...

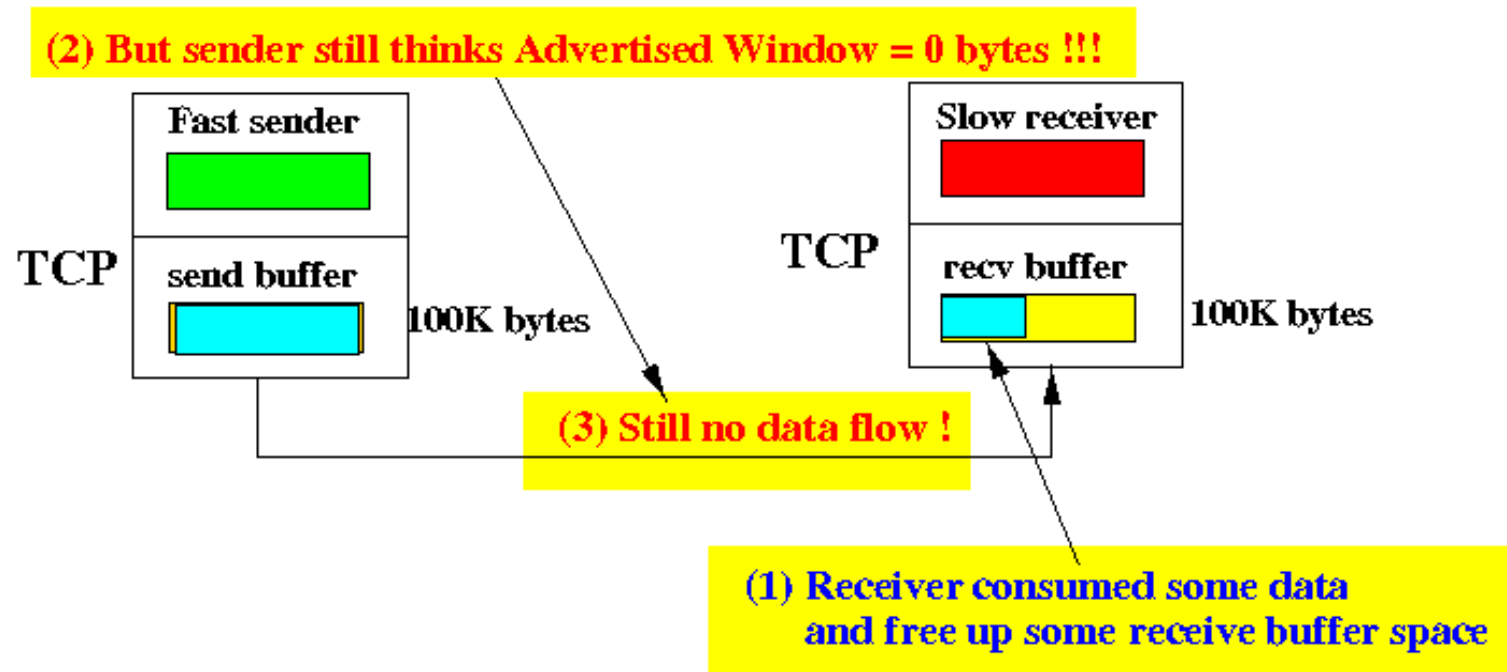


Unforeseen consequence of Flow Control:

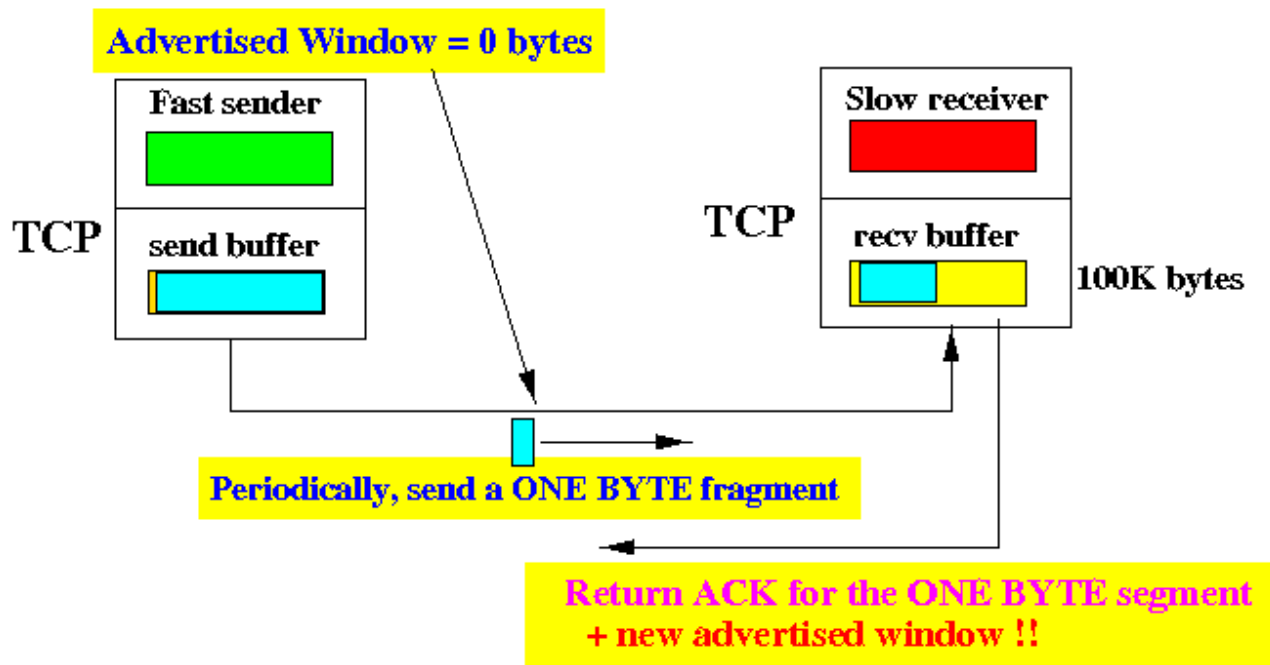
- Since the **advertised window size = 0** at the sender, the sender can *not* send any packet...
- **Consequently, the receiver may *not* send any packet to the sender**
- **Consequently, the Advertised Window Size of the sender will remain ZERO**



- We have a **"dead-lock"** situation:



- If the **advertised window size = 0** at the **sender** and the sender has some data in the *send buffer*, the sender will *periodically* send a *one byte* TCP segment to the receiver to **trigger a response** from the receiver:



The acknowledgement for the byte size probe TCP segment will contain the new (non-zero) value of the advertised window size that the sender can use to pace its transmissions



Short History of TCP and Congestion Control

- **1974:** Transmission Control Protocol - TCP was designed by
- **Vint Cerf, Yogen Dalal and Carl Sunshine (Stanford)**
- **One Indian in the list (IIT Bombay Btech)**
- **RFC 675**
- **1983:** TCP adopted as Internet's transport protocol
- **October 1986:** Van Jacobson reported in his classic paper that the **throughput** on a link between Laurence Berkeley Lab (LBL) and UC Berkeley (3 hops away) dropped to **0.125%....**

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and two IMP hops) dropped from 32 Kbps to 40 bps. We were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why



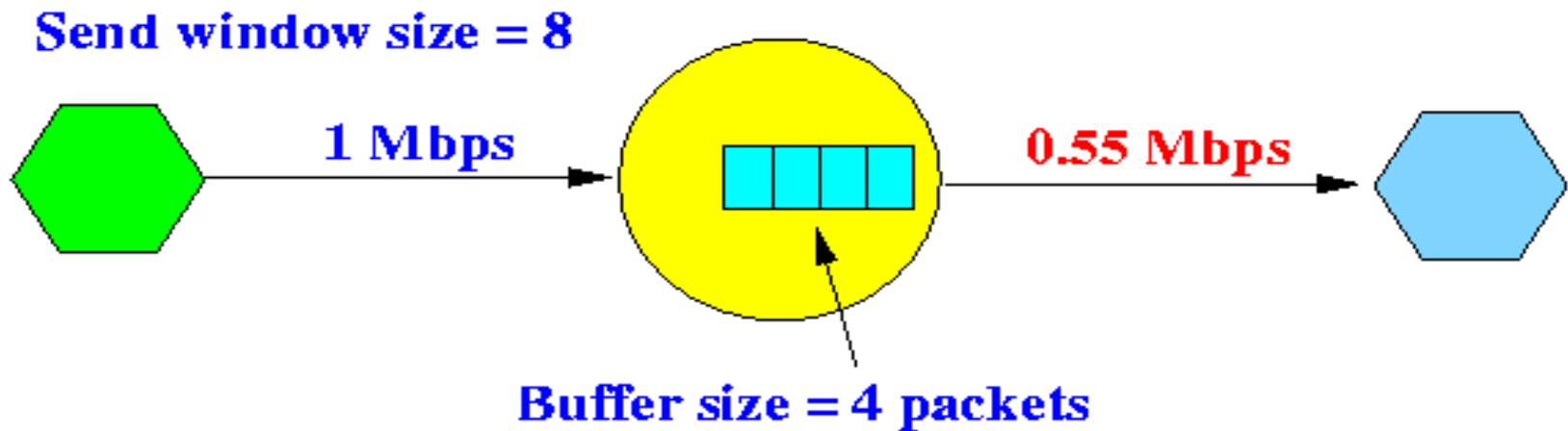
From paper ...

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and two IMP hops) dropped from 32 Kbps to 40 bps. We were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why



The throughput collapse was caused by the *lack of congestion control* in the TCP protocol

- Congestion, Buffer overflow and Packet Drop
- Example network:
-



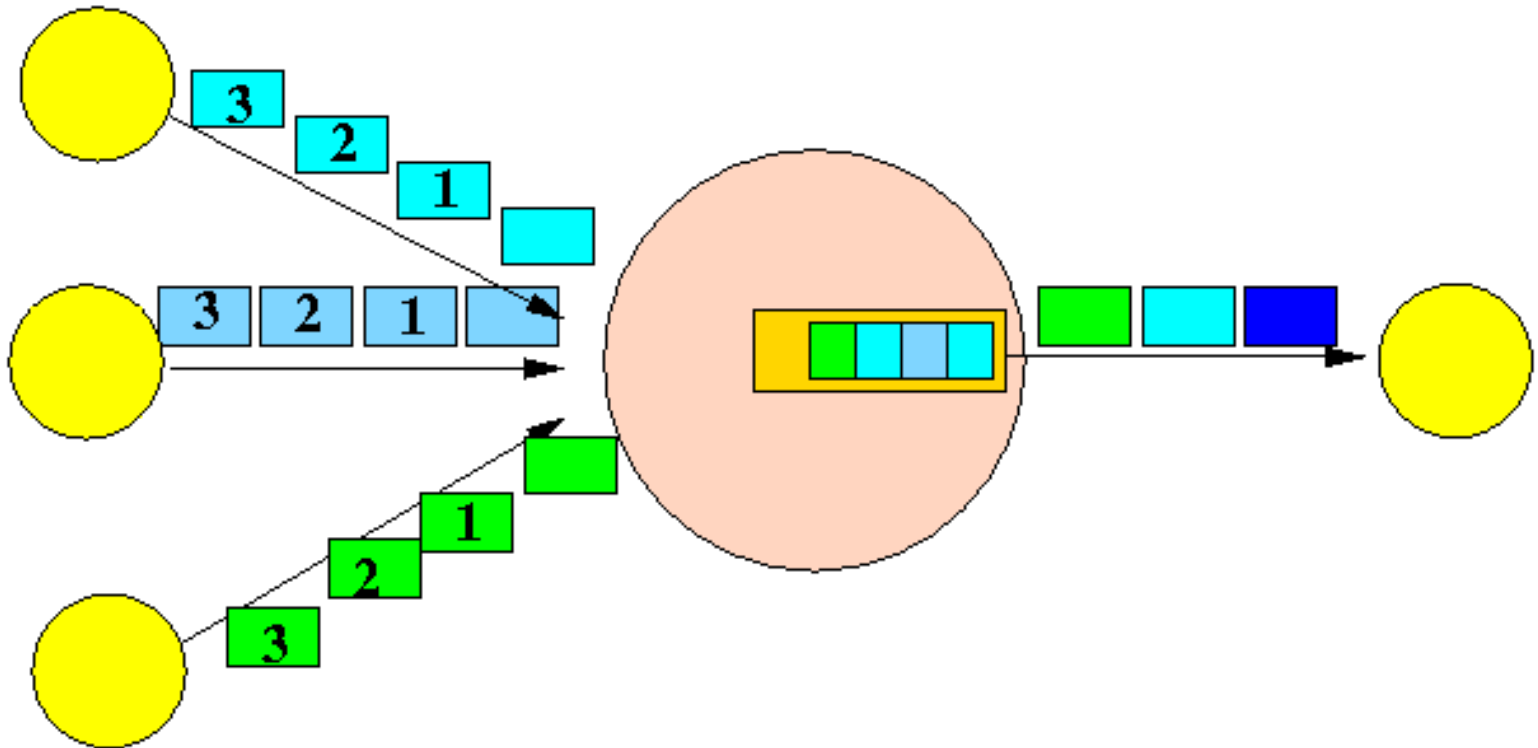
Effect of network congestion on TCP behavior

- Remember that TCP is a **reliable** protocol which means that TCP will **retransmit** packets that the source believe **were lost**.
- Suppose a number of sources use TCP to transfer a large amount of data, and their path uses a **common output link**.
- This situation will result in congestion:



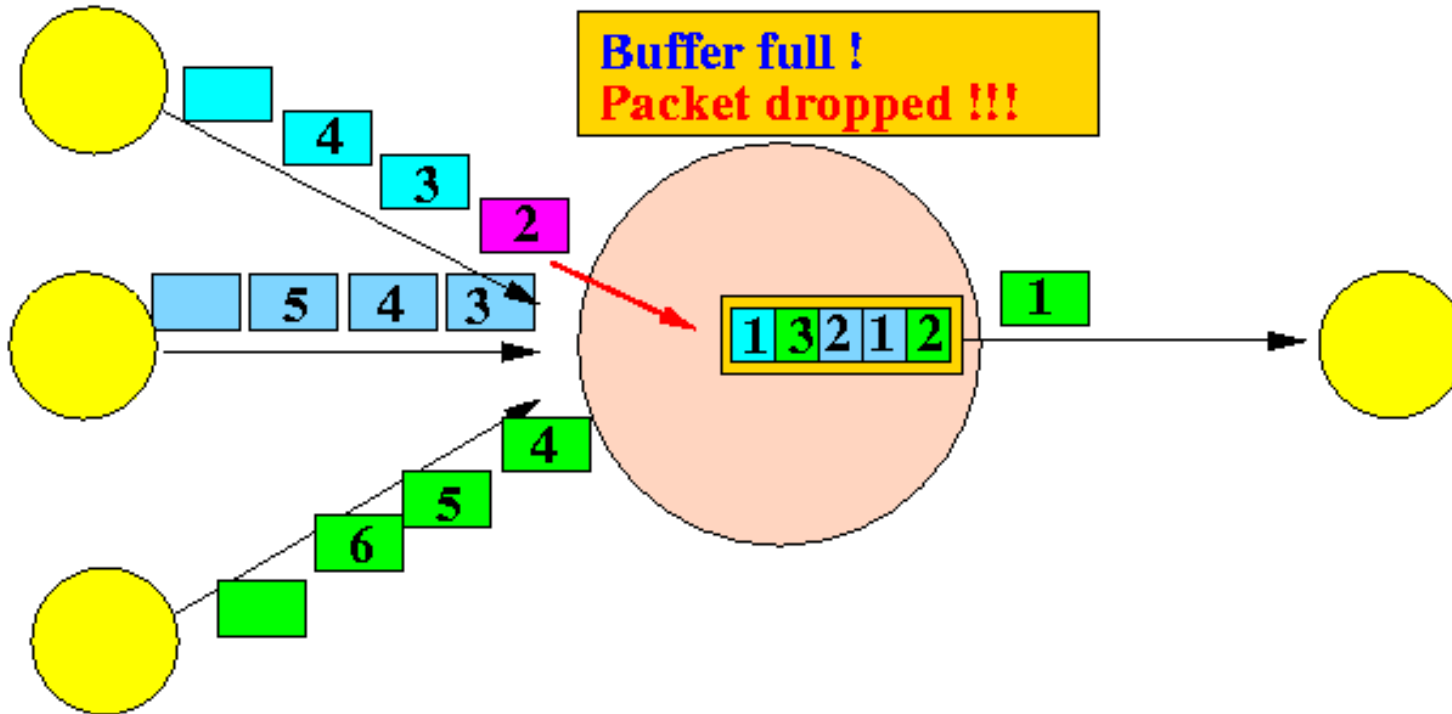
1

- The buffer of the output link will fill up gradually because the **fill rate** is *higher* than the **drain rate**:



2

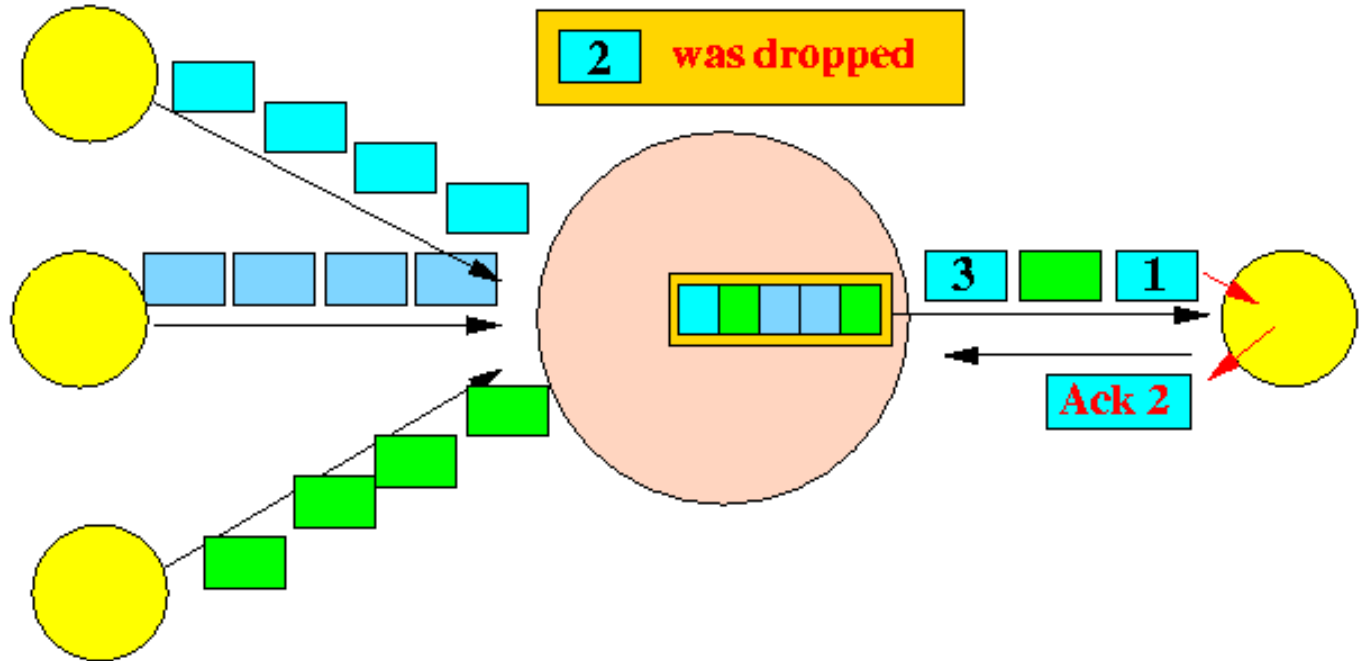
- Sooner or later, the buffer of the output link is full and **arriving packets are dropped**:



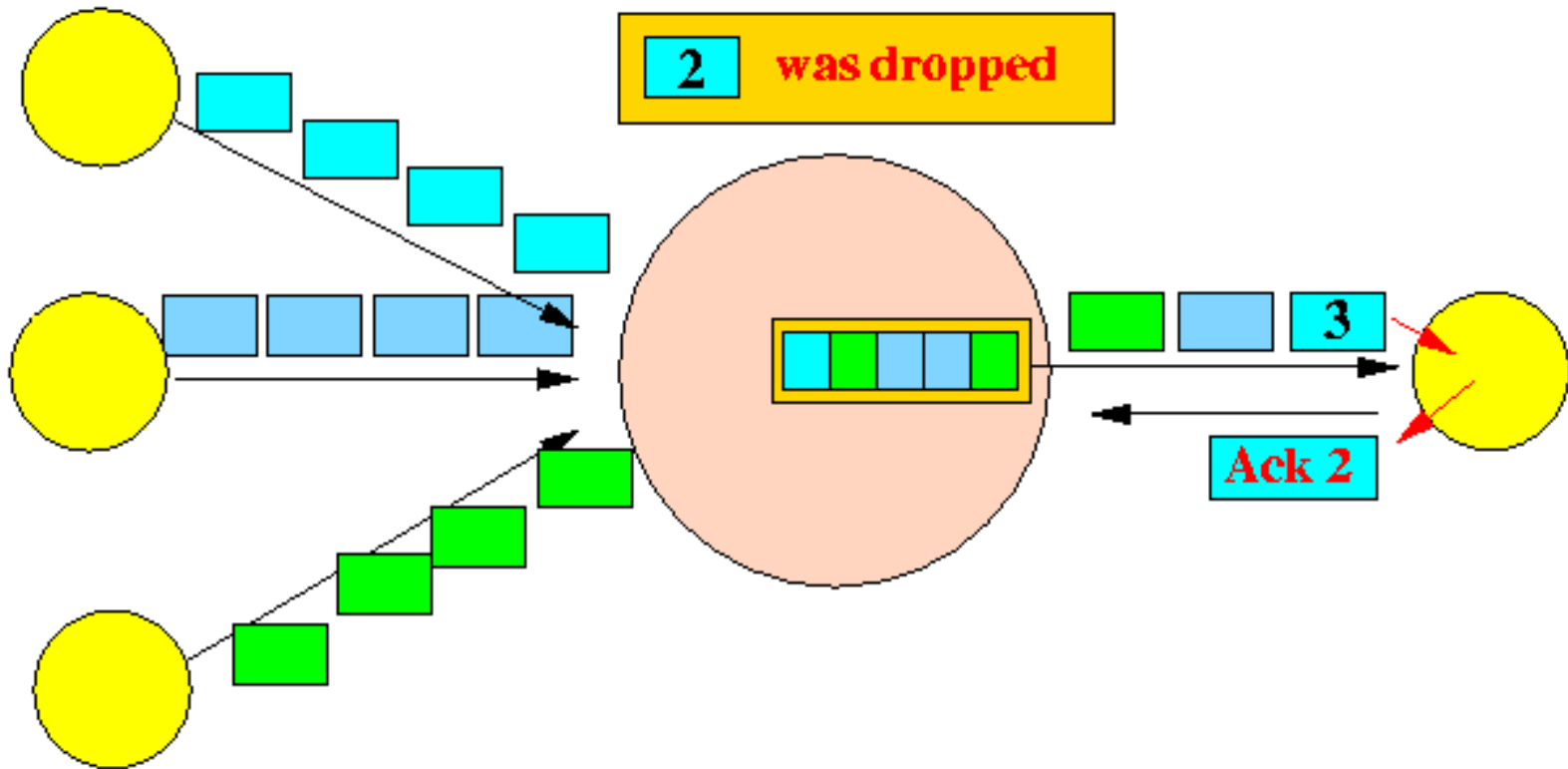
In this example, we assume that **packet 2** of the top stream is dropped

Traditionally: TCP uses "ACK n" to mean: all packets upto but *not including* n are received

When packet 1 is received, the ACK 2 is returned

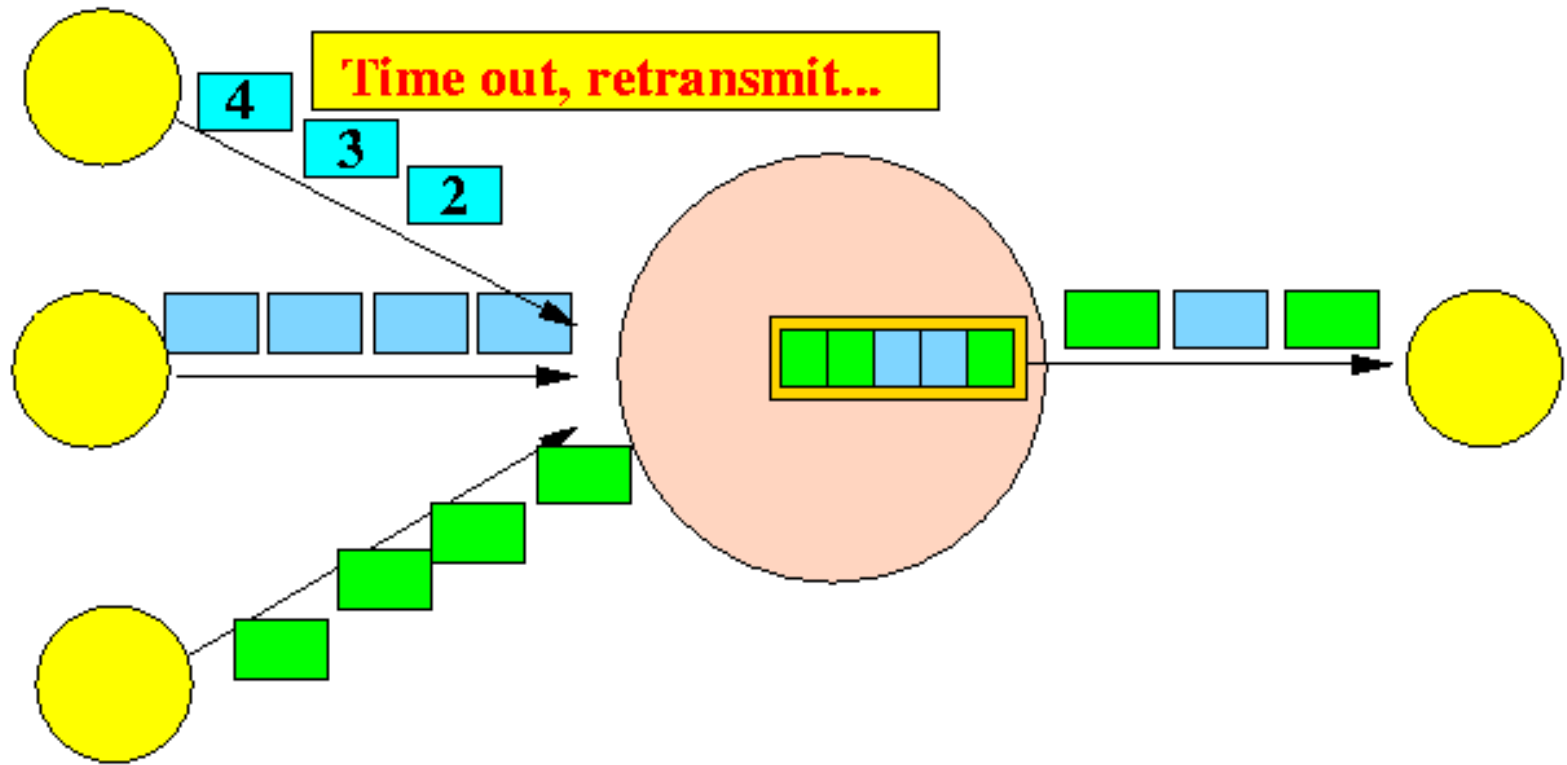


- When later packets (for example, **packet 3**) are received, the receiver will always return **ACK 2**
- It cannot send **ACK 4**, because that will make the sender omit **packet 2**...

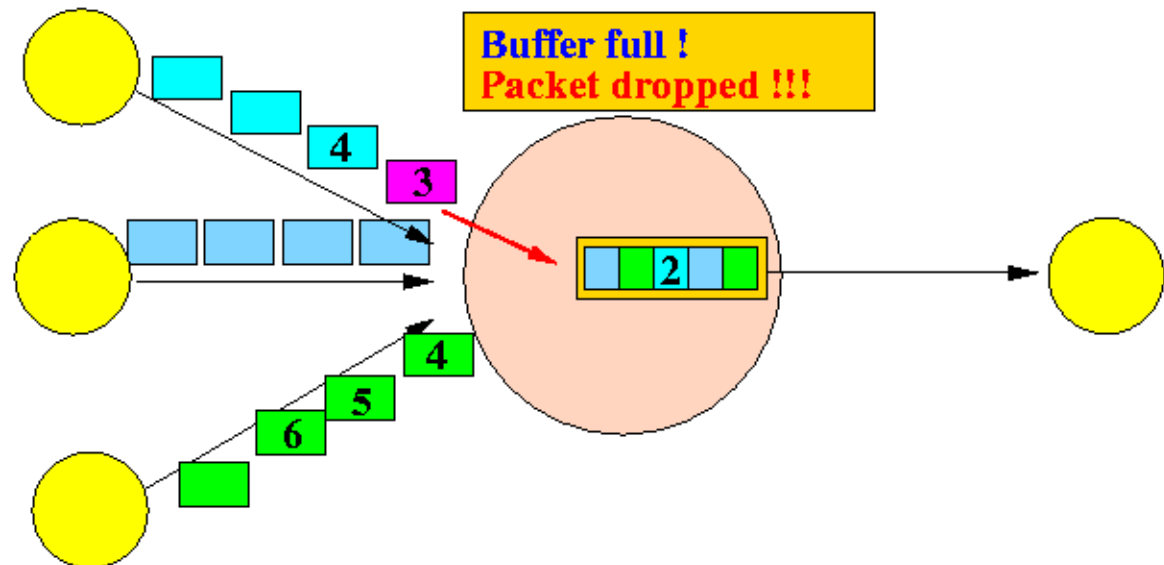


Repeated transmissions -

- After the sender times out, it will retransmits **all packets starting with packet 2** again (because due to congestion - long delay - it is likely that packets 3, 4, etc will also time out !)



- If the router is still **congested** (i.e., the buffer is **full**) →
 - Some of the packets of the retransmissions are likely to be **dropped** by the congested router



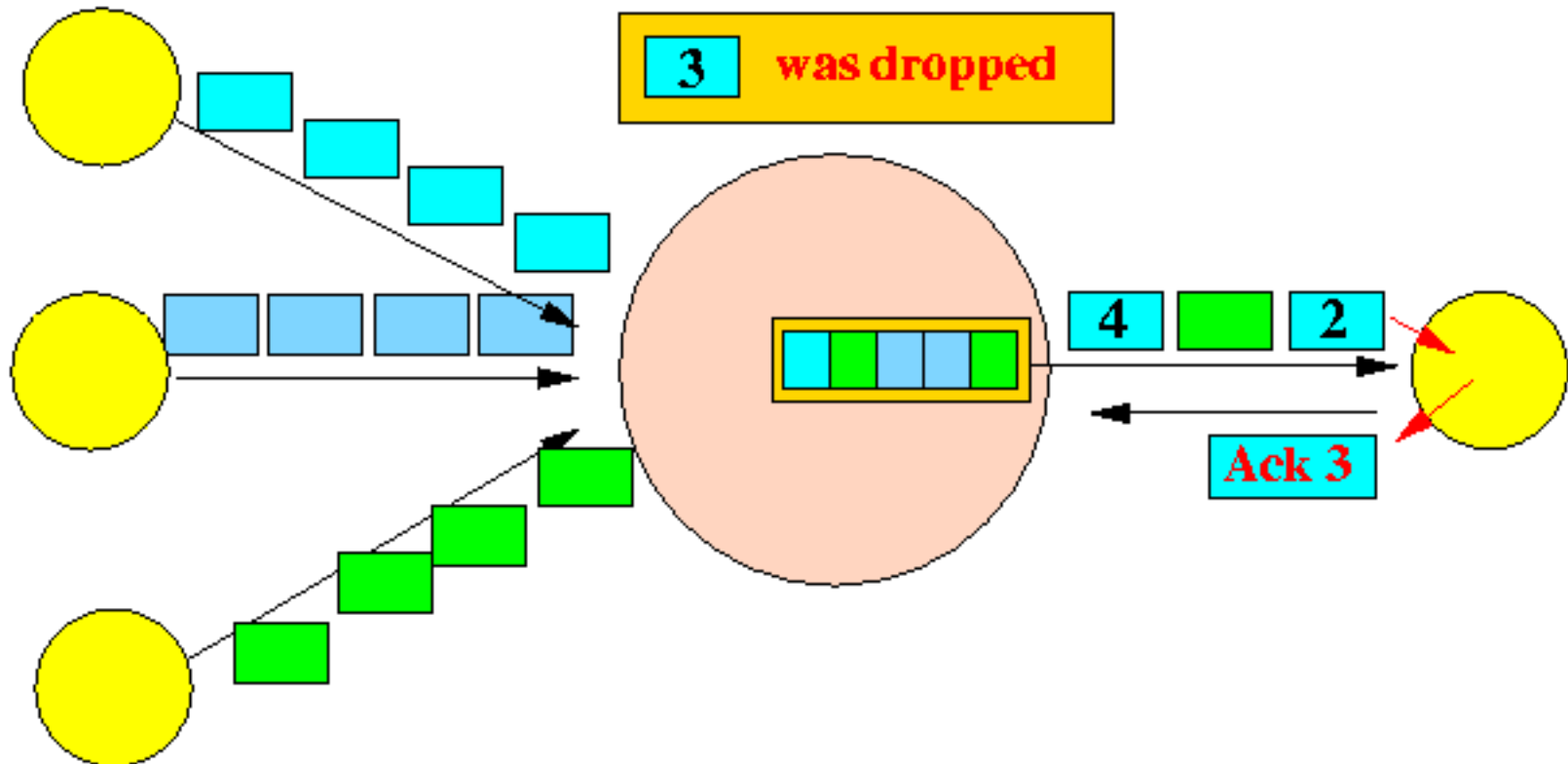
Possibility -

Maybe the first packet of the top stream got lucky and found an empty buffer, but the second packet (**packet 3**) was not as lucky and got dropped

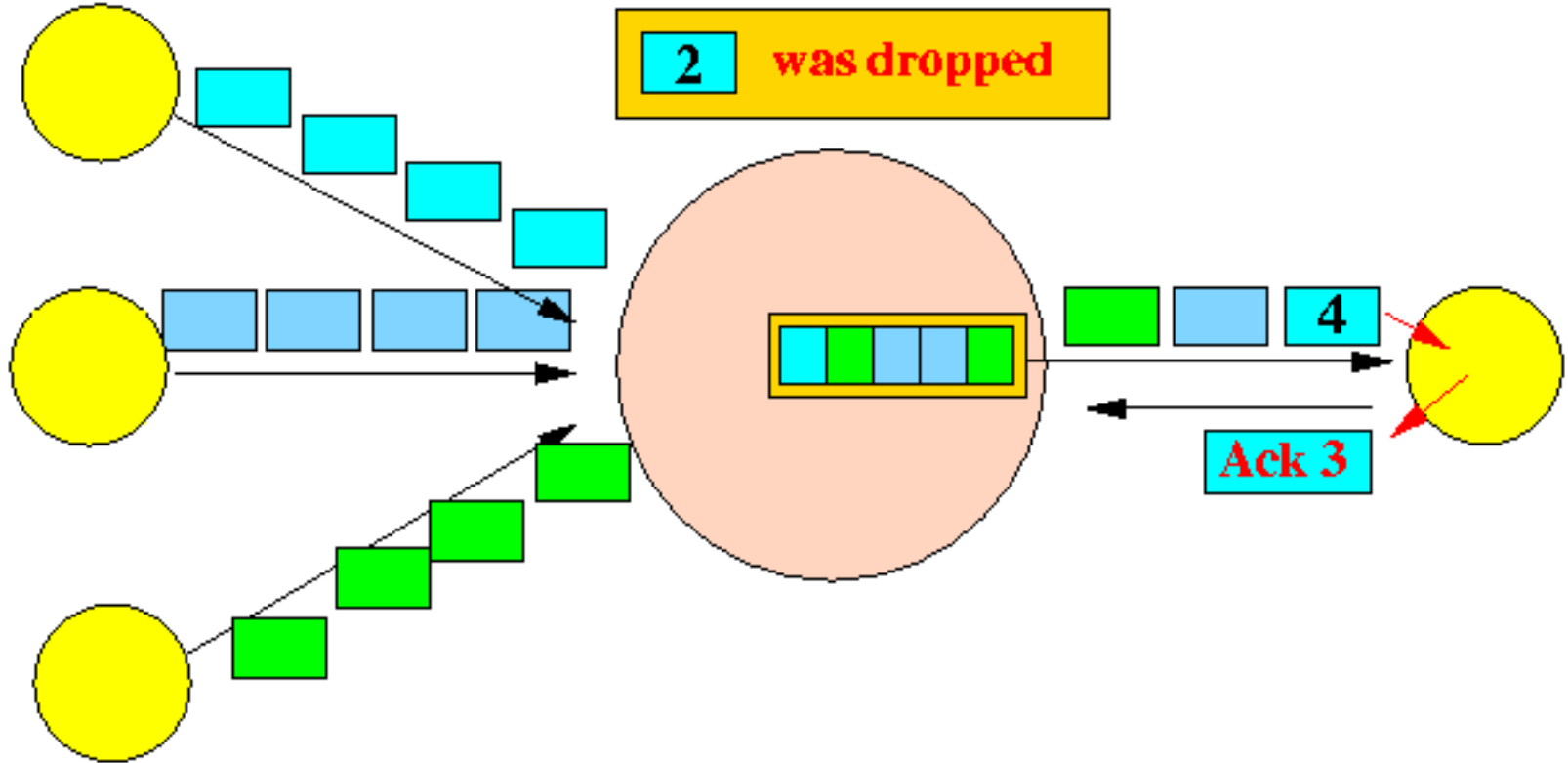
When **packet 2** is received, the **ACK 3** is returned:



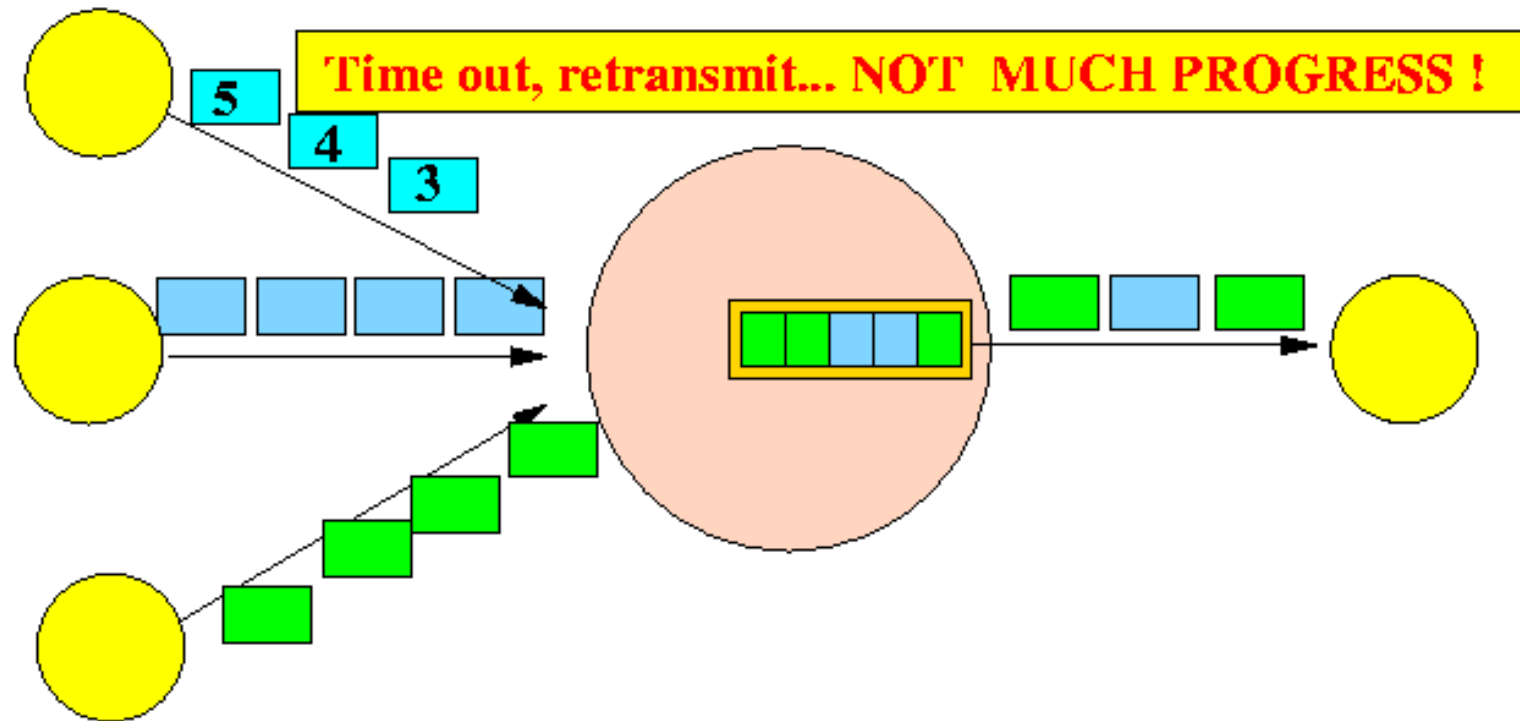
Packet 3 could not get place



- When later packets (for example, **packet 4**) are received, the receiver will always return **ACK 3**:



- After the sender times out, it will retransmits **all packets starting with packet 3** again (because due to congestion - long delay - it is likely that packets 4, 5, etc will also time out !)



And So on ...

- We have only focussed on one flow
- This is likely to happen to ALL flows
- When the buffer overflows



Result of congestion:

- The sender sends many packets
- But sender **makes *very little progress***
- Sender **retransmits** many packets (causing more congestion and more packet drops)
- If we do nothing, we have a **"network collapse"...**



TCP Congestion Control

- This **congestion problem** was recognized and studied mainly by **Raj Jain** (when he was working at Digital Equipment Corporation - **DEC**)
- And later by **Van Jacobson**
- They both recognize that the solution to the congestion problem was:
- The **source** must be **aware** of the current state of the network path and **adjust** its transmission rate accordingly.



- The difference was that R Jain (working at DEC) has been using DEC's network architecture and protocol (**DECNET**)
- He studied the congestion problem **very thoroughly** from a **theoretic** point of view as he experimented and developed many different ways to allow the source to become aware:
 - **Using the "DEC bit" set by the routers to signal congestion**
 - **Using timeout as a congestion signal**
 - **And even using average delay as a congestion signal**



But...

- NOTE: TCP is **not perfectly fair**.
- It is **biased** towards connections between source/destination pairs with **small RTT** (these connections get a relatively larger share of the available bandwidth)



"TCP-friendliness"

- TCP traffic comprises a very large portion of the traffic on the Internet,
- **New transport protocol** that researchers propose for use on the Internet **must be politically correct** towards TCP !!!
- In other words: any new transport protocol must not **compromise** TCP's ability to obtain a fair share of network resources (bandwidth)
- **Note:** it is quite easy to bully TCP and rob it's share of bandwidth using **UDP**.... just transmit as fast as you can with the UDP protocol and TCP will "automatically" backoff and use **less bandwidth** (because TCP detected congestion and reduce the source's transmission rate)



Motivation

- **Goal: Transmit a (large) file from point A to point B as FAST as possible**
- **But – the quickest way to transmit data is not always transmitting the data as fast as you can....**



Analogy

- It may help to understand this unintuitive fact by observing **traffic on the highway**:
- Traffic on the highway will actually **move faster** if the traffic load is **high enough but not too high**.
- When the **traffic load is higher than what the highway can handle**, a **congestion will build at some *bottleneck point*** (usually an on or off ramp) and causes traffic on the **whole** highway to slow down to a trickle....
- **Key:** It is **beneficial** to a flow to **restrict** its own transmission rate **if** the current transmission rate is more than the network can handle.



This “Sacrificial Act” of restricting one's own transmission rate is known as congestion control

- If the network's capability can handle **more** traffic, then it is "natural" than one would **increase** one's transmission rate.
- **So, congestion control mechanism** also includes the ability to **increase** one's transmission rate (although it's kind of unintuitive to call this action "congestion control"... since it would initiate congestion
- Congestion control is one of a few application of **communism** that will actually work
- Because it benefit **oneself** if one will do it as long as everyone else will do it also – It may fails here also because we cant assure what others will do..



Congestion control

- The **basic principles** of congestion control is simple:
- **Decrease** your transmission rate if you detect **congestion**
- **Increase** your transmission rate if you detect **no congestion**
- *How fast* should you **decrease** your transmission rate if you detect **congestion**
- *How fast* should you **increase** your transmission rate if you detect **no congestion**



Designing congestion control methods

- You need to solve **2 problems** when you are **designing a congestion avoidance or control methods**:
- **How to *detect* an onset or presence of congestion**
- **How *fast* do you:**
 1. Decrease transmission rate when you do detect congestion.
 2. Increase transmission rate when no congestion is detected



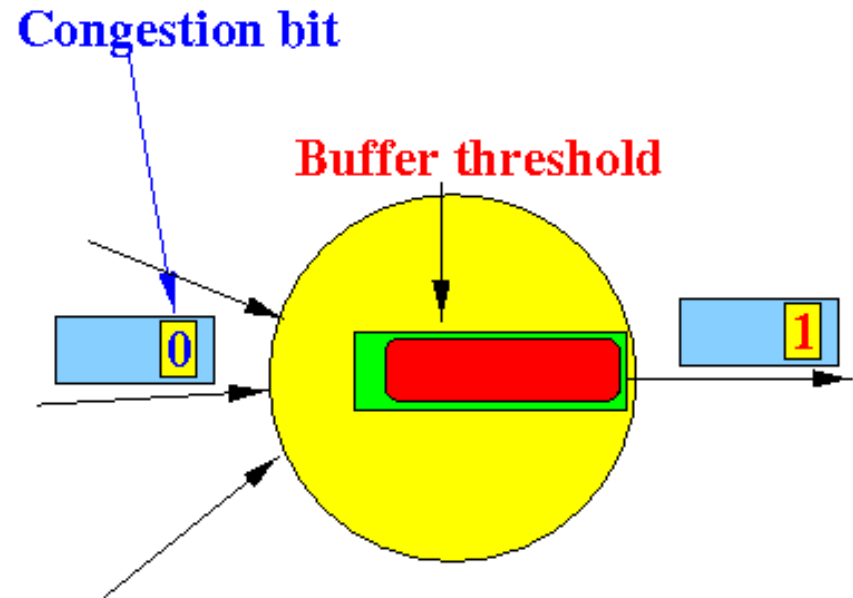
Detecting Congestion

- **Network congestion** is detected by using **one or more "signals"** from the network...
- **Type of signals** used to detect congestion:
- *Explicit signals*:
- An explicit signal is a value that is sent **inside a packet**



Example

- In the **DECBit method**, the packet header contains a **congestion bit**
- When the packet is sent by the sender, this bit is **RESET**
- When the packet passes through a **congested router**, (the queue length exceeds a certain threshold) the router will **set the congestion bit**



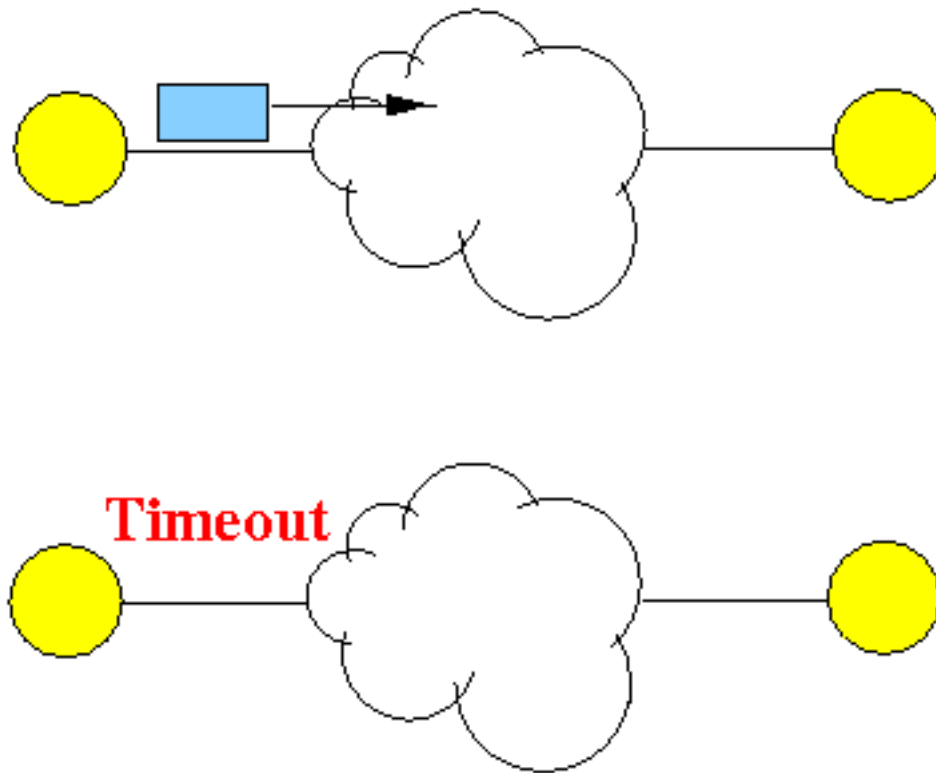
Note

- *Explicit signals* requires the **assistance** of routers and therefore, it is **NOT useful practically** because present day Internet routers **do not** flip bits in packets...

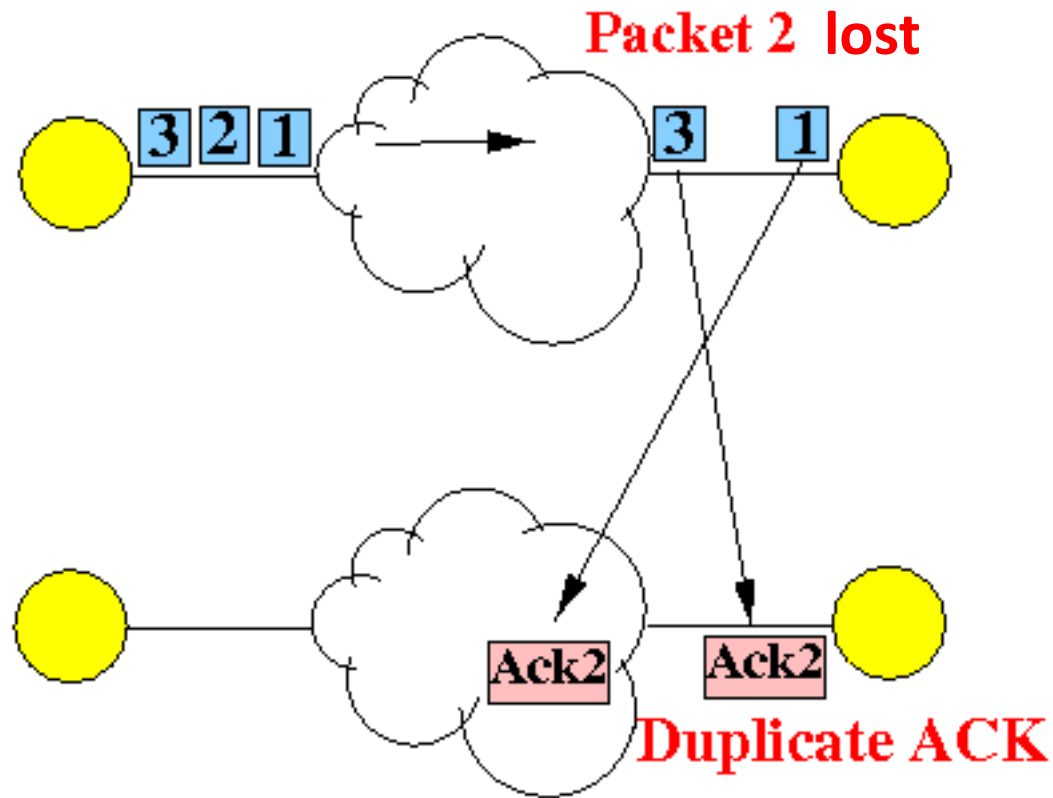
Implicit signals:

- An implicit signal is a value that is **derived (concluded)** from events and it is not conveyed through packets
- **Examples: Packet loss**
- Congestion is detected when the sender detects that some packet was lost

Packet loss can be detected through **timeout**:

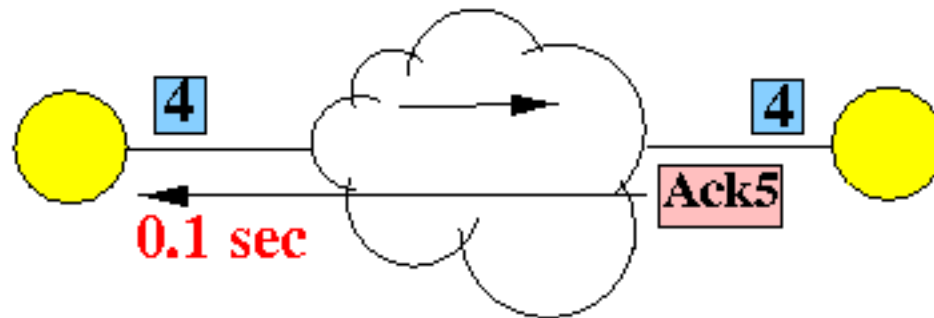
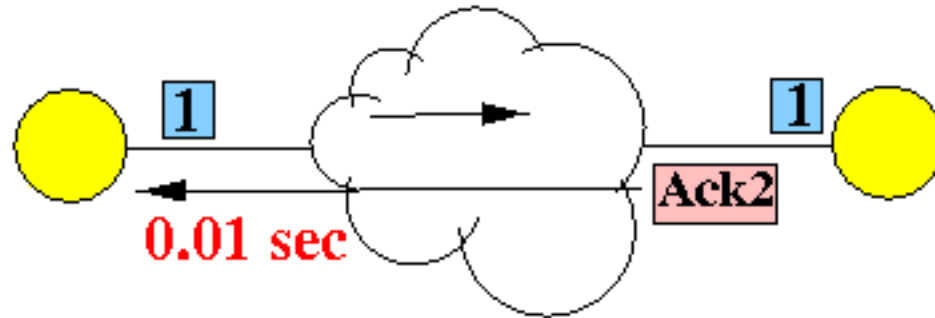


Or by duplicate ACKs:



Packet delay

- Sender measures the time to receive ACK.
- Time exceeds a certain threshold $\rightarrow$ Congestion.



Implicit

- Due to the fact that routers in the Internet **do NOT** provide **explicit signals**, we will only consider **congestion control** using *implicit* signals

Reliability of the Packet Loss Signal

- When **a packet is lost**, it's a **sure sign** that the network path is congested
- **However**, *packet loss* is the **worst possible feedback signal** that you can use !!!
- The **purpose of congestion avoidance** is to *avoid* congestion
- The **loss of a packet** is in fact an indication that the network is *already* congested - **some buffer was full** at some router
- What we want to achieve is to *avoid* the buffer being **filled up...**



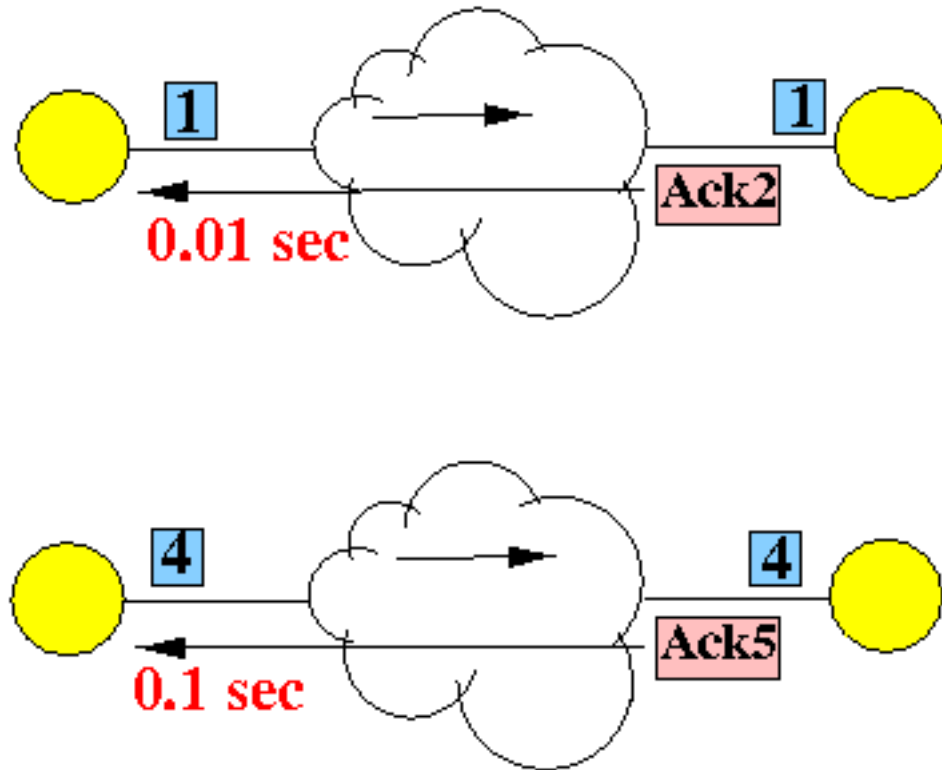
We need prevention

- It is somewhat akin to a medical situation where the patient is having a **heart attack**:
- What we really want is to **prevent** the heart attack
- What we are really doing is: **fix** the patient when the heart attack is happening...
- **NOTE:**
- Because "it was too late", the **congestion control measure** taken by the sender **when packet loss is detected** will have to be **VERY DRASTIC** (discussed later)



Reliability of the End-to-end Delay Signal

- The **End-to-end delay** is a far better indication than packet loss:
- By measuring the end-to-end delay, the source can detect an **impeding collapse** and take measure to reduce the traffic before the collapse happens:



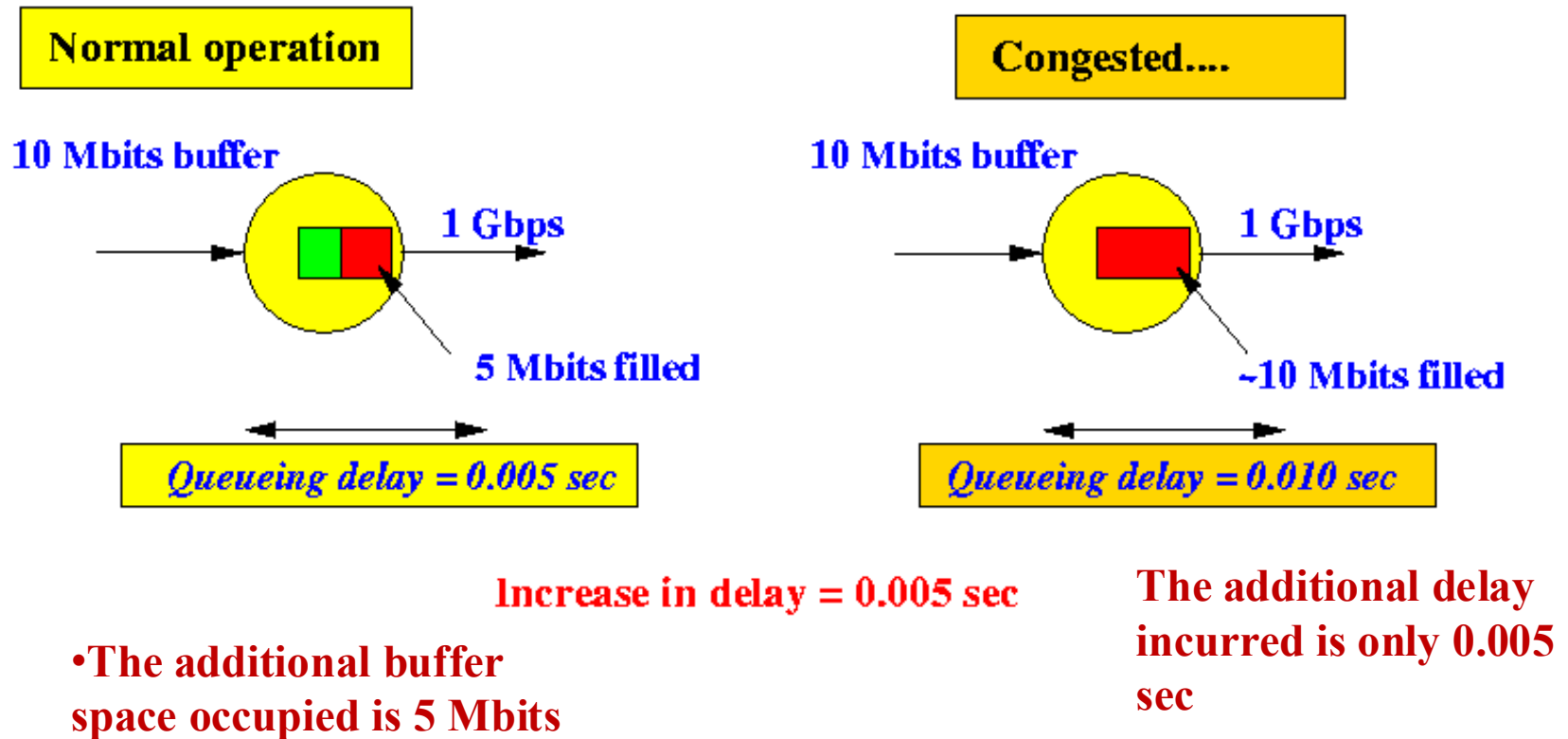
But the **End-to-end delay** signal is also **harder to use**

- It is much more difficult to determine if the **buffer** of a router is about to **overflow** by observing the end-to-end delay...
- **Specifically:**
- You cannot accurately tell from the increase in the delay how **bad** the situation is.



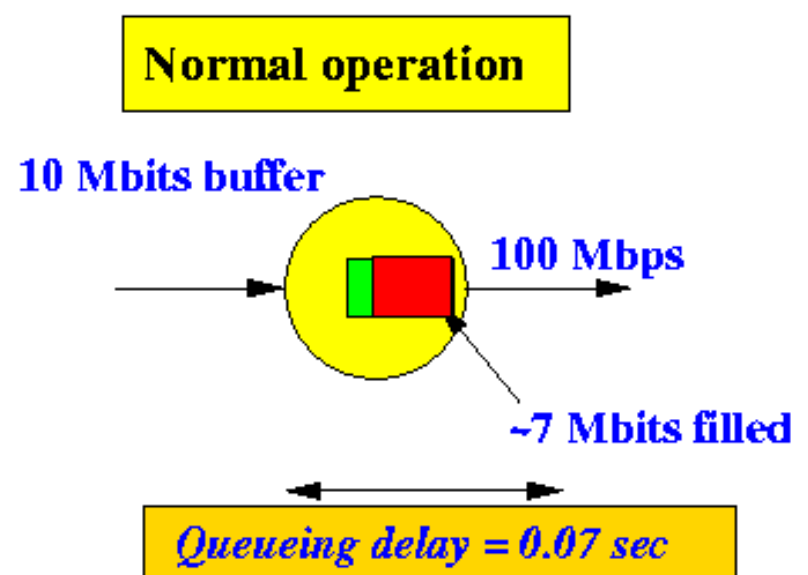
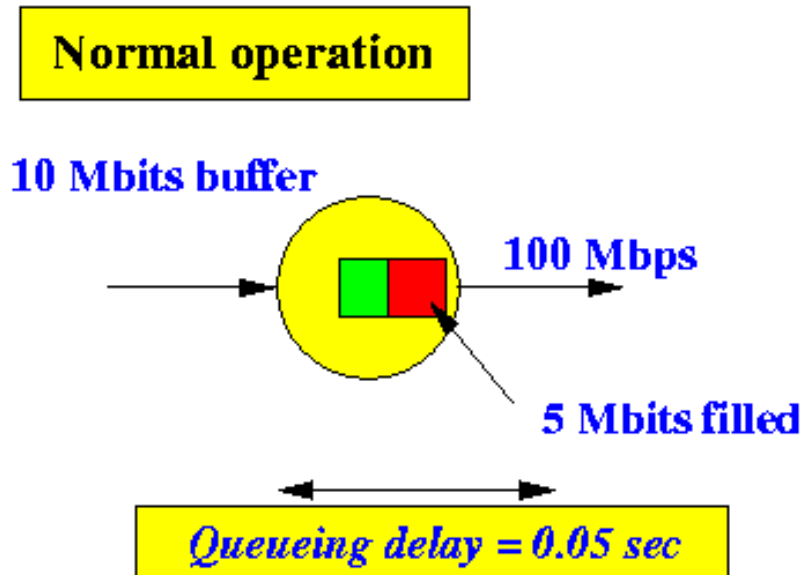
Example:

- When the **drain rate** is **high** (1 Gbps), a small increase in delay can correspond to a buffer overflow:



Now consider a *lower* drain rate:

But the additional delay incurred is 0.02 sec



Increase in delay = 0.02 sec

The additional buffer space occupied is **ONLY 2 Mbits**



Conclusion

- Absolute increase in delay is not a good indicator
- However, a study on delayed based congestion control methods can be found in Raj Jain's works